

Analyse des résultats expérimentaux — RL-in-GA pour le problème du sac à dos

RALAIVAO Niaiko Michaël — Master Recherche, ENI / LIMAD / GLORIA

Septembre 2026

Table des matières

1	Organisation des résultats	1
2	E0 — Calibration (instances disjointes des tests)	2
3	E1 — KP 0/1 générés (54 instances × 7 variantes × 10 exécutions)	2
3.1	Ce que montre la politique apprise (QR5)	3
4	E2 — KP 0/1 de référence : instances_01_KP et Jookan et al. (31 instances × 6 variantes × 10 exécutions)	4
5	Cas d’usage réels (données synthétiques illustratives ; 5 exécutions par variante d’AG)	5
6	E3 — Sac à dos multidimensionnel, OR-Library (58 instances × 6 variantes × 10 exécutions)	6
7	E5 — Passage à l’échelle (classes UC et SC, n = 100 à 10 000, 3 graines)	6

1 Organisation des résultats

Les résultats sont regroupés par campagne ; chaque figure porte un numéro stable (préfixe figNN_) et chaque tableau existe en CSV, Markdown et LaTeX dans `results/analyse/<campagne>/`, pour une intégration directe dans le manuscrit.

Campagne	Question	Tableaux (results/analyse/...)	Figures (results/figures/...)
E0 — calibration	quels réglages ?	exp0_pilot/	fig00_*, fig05_*, fig06_*
E1 — KP générés	QR1, QR2, QR3, QR5	e1_kp01_generes/	fig01_e1_*, fig04_politique_actions
E2 — KP de référence	QR1, QR3, QR4	e2_kp01_benchmarks/	fig02_e2_*
E3 — MKP	QR3 (transfert inter-problèmes), QR4	e3_mkp/	fig03_e3_*
E5 — passage à l’échelle	QR4	e5_scalabilite/scalabilite	fig07_scalabilite
Cas d’usage	utilité pratique	cas_usage/resultats_cas_usage	passage_captures

Lecture des indicateurs. L’écart est $100 (f_{\text{ref}} - f) / f_{\text{ref}}$ en % (0 = optimum). Les écarts sont faibles en valeur absolue (quelques centièmes de %) parce que l’AG, grâce à la réparation gloutonne, est déjà un bon solveur ; ce sont les **différences relatives** entre méthodes et leur **significativité**

qui importent. Les comparaisons suivent les recommandations de DEMŠAR (2006) et DERRAC et al. (2011) : test de Friedman suivi du test post hoc de Nemenyi pour le classement global, test de Wilcoxon apparié avec correction de Holm pour les comparaisons deux à deux. $V/E/D$ = nombre d’instances où la méthode cible gagne / fait jeu égal / perd (Mann–Whitney, $\alpha = 0,05$) ; $A_{12} > 0,5$ signifie que la cible obtient plus souvent un écart plus faible (taille d’effet de VARGHA et DELANEY (2000)).

2 E0 — Calibration (instances disjointes des tests)

Réalisée sur 6 instances (SC, ASC, UC ; $n = 500$; graines 900+), selon la démarche de réglage des paramètres des algorithmes évolutionnaires décrite par EIBEN et al. (1999) :

1. **Tous les contrôleurs adaptatifs** (aléatoire, UCB1, QL en ligne) divisent par ≈ 4 l’écart de l’AG standard sur SC/ASC, mais le **QL en ligne ne dépasse pas la sélection aléatoire** (écart moyen 0,0182 % contre 0,0158 %).
2. Parmi les **9 configurations fixes**, **UX + BF3** est de loin la meilleure (SC : 0,0000 % ; ASC : 0,0073 %) et **1P + BF1** (l’AG standard) la pire (SC : 0,1105 %) — un facteur > 10 : **le choix des opérateurs compte**, et il existe donc une marge pour l’apprentissage, ce qui est la prémisse de la sélection adaptative d’opérateurs (DA COSTA et al., 2008 ; KARIMI-MAMAGHAN et al., 2022).
3. Choix de la **récompense** (Q-table entraînée sur UC/WC/SS puis testée) : l’amélioration relative seule (R1) donne la meilleure politique (SC 0,0000 %, ASC 0,0149 %) ; l’ajout du taux de succès des descendants (SC 0,0237 %) ou d’une pénalité de diversité (SC 0,0395 %) la dégrade. → **R1 retenue**. Ce résultat est cohérent avec les récompenses fondées sur l’amélioration de la qualité, les plus fréquentes dans les revues sur l’apprentissage par renforcement au service des algorithmes évolutionnaires (LI et al., 2025 ; SONG et al., 2023).

3 E1 — KP 0/1 générés (54 instances $\times$ 7 variantes $\times$ 10 exécutions)

Les instances sont générées selon les classes de PISINGER (2005) ; les contrôleurs comparés sont l’AG standard (GOLDBERG, 1989 ; HOLLAND, 1975), la sélection aléatoire, le bandit UCB1 (AUER et al., 2002) et le Q-learning (SUTTON & BARTO, 2018 ; WATKINS & DAYAN, 1992), en ligne ou pré-entraîné.

Tableau 1 — Écart moyen à l’optimum (%) (optimum prouvé par programmation dynamique (BELLMAN, 1957 ; KELLERER et al., 2004a)) :

Classe	AG standard	Aléatoire	UCB1	QL en ligne	QL pré-entraîné	QL pré-entr. + $\varphi(I)$	Oracle fixe
UC	0,0048	0,0028	0,0024	0,0031	0,0019	0,0031	0,0021
WC	0,0063	0,0039	0,0036	0,0039	0,0037	0,0035	0,0033
SS	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000
ISC	0,0053	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000
SC	0,1114	0,0258	0,0275	0,0275	0,0158	0,0258	0,0086
ASC	0,1016	0,0341	0,0321	0,0319	0,0232	0,0309	0,0156
Moyenne	0,0382	0,0111	0,0109	0,0111	0,0074	0,0105	0,0049
Rang moyen (Friedman)	6,05	4,08	3,96	4,08	3,10	4,09	2,63

Test de Friedman : $\chi^2 = 130,15$, $p = 1,2 \cdot 10^{-26}$ ($N = 54$ instances, différence critique de Nemenyi CD

= 1,23). Pour comparaison, le **glouton** fondé sur l’ordre des efficacités (DANTZIG, 1957) laisse un écart moyen de 0,34 % sur SC et ASC, soit **plus de 10 fois** celui des variantes adaptatives de l’AG.

Tableau 2 — Tests appariés (Wilcoxon sur les 54 écarts moyens par instance, correction de Holm) :

Comparaison	Écarts (%)	p (Holm)	A_{12}	V/E/D
QL pré-entraîné vs AG standard	0,0074 vs 0,0382	$< 10^{-10}$	0,74	28/26/0
QL pré-entraîné vs aléatoire	0,0074 vs 0,0111	0,0005	0,59	9/45/0
QL pré-entraîné vs UCB1	0,0074 vs 0,0109	0,003	0,58	10/44/0
QL pré-entraîné vs QL en ligne	0,0074 vs 0,0111	0,002	0,58	7/47/0
QL pré-entraîné vs oracle fixe	0,0074 vs 0,0049	0,003	0,42	0/45/9
QL en ligne vs AG standard	0,0111 vs 0,0382	$< 10^{-10}$	0,71	24/30/0
QL en ligne vs aléatoire	0,0111 vs 0,0111	1,00 (n.s.)	0,51	0/53/1
QL en ligne vs UCB1	0,0111 vs 0,0109	1,00 (n.s.)	0,51	1/51/2

Effet de la taille (classes SC et ASC) : l’écart de l’AG standard croît de 0,046 % ($n = 200$) à 0,167 % ($n = 1\,000$) ; celui du QL pré-entraîné passe de 0,005 % à 0,025 %, contre 0,043 % pour le QL en ligne et l’aléatoire à $n = 1\,000$. **L’avantage du pré-entraînement augmente avec la taille** : les instances de 500 et 1 000 objets sont hors de la distribution d’entraînement (≤ 200 objets).

Vitesse : le QL pré-entraîné atteint son meilleur résultat plus tôt (génération 48,8 en moyenne, contre 56,9 pour l’AG standard et 50,5 pour l’aléatoire) ; le surcoût de l’agent est négligeable ($\approx 4,6$ s contre 4,0 s par exécution, à n moyen de 567).

Méthodes exactes : la PD résout toutes les instances d’E1 en $\leq 3,4$ s ; la PLNE (solveur HiGHS (HUANGFU & HALL, 2018), appelé depuis SciPy (VIRTANEN et al., 2020)) trouve l’optimum sur les 54 instances mais **ne prouve pas l’optimalité** dans la limite de 30 s pour 12 instances corrélées (SC, ISC, ASC de 500–1 000 objets), confirmant leur difficulté pour le *Branch-and-Bound* (MARTELLO & TOTH, 1990 ; PISINGER, 2005).

Figures associées : fig01_e1_ecart_par_classe (écarts et IC 95 %), fig01_e1_rangs_friedman, fig01_e1_boites (distributions), fig01_e1_convergence (courbes et diversité), fig01_e1_succes_echecs, fig01_e1_qualite_temps, fig04_politique_actions.

3.1 Ce que montre la politique apprise (QR5)

La figure fig04_politique_actions donne la fréquence des 9 actions par tiers de la recherche :

- **QL en ligne et UCB1** : distributions **quasi uniformes** (≈ 11 % par action dans toutes les phases). En 200 générations, ils n’identifient pas de préférence : leur comportement est en pratique celui de la sélection aléatoire — ce qui **explique** l’égalité statistique avec GA-RAND (H2).
- **QL pré-entraîné** : préférence nette pour le **croisement uniforme** — UX + SWAP (40 %) et UX + BF3 (17–30 %) en début de recherche, UX + BF3 (46–54 %) et UX + SWAP (33–39 %) en fin de recherche — soit exactement la famille de l’oracle (UX + BF3), **découverte sans la connaître** — ce qui illustre l’intérêt de la configuration dynamique d’algorithmes apprise hors ligne (BENJAMINS et al., 2024 ; BIEDENKAPP et al., 2020) —, sur des instances

faciles, et appliquée aux classes difficiles. En phase intermédiaire, la répartition est presque uniforme : ces états ont été peu visités pendant l’entraînement (valeurs Q quasi nulles), ce qui explique l’écart résiduel avec l’oracle.

- $\phi(I)$ n’apporte rien ici : avec 81 états au lieu de 27 pour un budget d’entraînement comparable, chaque état est moins visité ; la politique est plus diluée (écart 0,0105 % contre 0,0074 %). C’est la « malédiction de la dimension » des représentations tabulaires, qui motive l’approximation de fonction (MNIH et al., 2015; SUTTON & BARTO, 2018).

4 E2 — KP 0/1 de référence : instances_01_KP et Jookan et al. (31 instances × 6 variantes × 10 exécutions)

Ce jeu réunit des instances de Pisinger et les instances difficiles de JOOKAN et al. (2022), conçues pour mettre en défaut les méthodes exactes.

Tableau 3 — Écart moyen à l’optimum (%) (optimums fournis avec les instances) :

Classe	AG standard	Aléatoire	UCB1	QL en ligne	QL pré-entraîné	Oracle fixe
UC (n = 100...2 000)	0,0345	0,0184	0,0221	0,0190	0,0222	0,0229
WC	0,1904	0,1161	0,1571	0,1381	0,0827	0,1144
SC	0,0232	0,0142	0,0001	0,0008	0,0001	0,0139
Jookan (n = 400, 1 000)	0,0012	0,0010	0,0010	0,0009	0,0008	0,0008
Moyenne	0,0407	0,0245	0,0294	0,0259	0,0174	0,0248
Rang moyen (Friedman)	5,00	3,19	3,71	3,11	2,81	3,18

Friedman : $\chi^2 = 40,07$, $p = 1,5 \cdot 10^{-8}$ ($N = 31$, $CD = 1,35$).

- Le **QL pré-entraîné** obtient la **meilleure moyenne** et le **meilleur rang** ; il bat significativement l’AG standard (p Holm = 0,001, 5/26/0). Face aux autres variantes adaptatives, les différences ne sont **pas significatives** sur ces 31 instances (p Holm $\geq 0,21$) : avec 5 instances par classe (tailles de 100 à 2 000 objets), la variance entre instances est forte, et les instances de Jookan sont résolues à moins de 0,003 % par toutes les variantes, ce qui laisse peu de place aux différences. Le gain le plus net du pré-entraînement porte sur la classe WC (0,083 % contre 0,116–0,157 % pour les autres variantes adaptatives).
- **L’oracle n’est plus le meilleur** : UX + BF3, choisi sur les instances de calibration générées, se classe au niveau de l’aléatoire sur ce jeu (0,0248 %). Une configuration fixe réglée sur une distribution ne se transfère pas nécessairement ; la **politique apprise**, qui varie les opérateurs selon l’état de la recherche, se transfère mieux (0,0174 %). Ce constat rejoint le théorème « pas de repas gratuit » : aucun réglage fixe n’est meilleur sur toutes les distributions d’instances (WOLPERT & MACREADY, 1997).
- **QL en ligne $\approx$ aléatoire** ($p = 1,00$), confirmant H2 sur un second jeu.

Le mur des méthodes exactes (instances de Jookan, $C = 10^8$). Pour les 8 instances de capacité 10^8 , la **programmation dynamique est infaisable** (vecteur de 10^8 réels $\approx 0,8$ Go, table de reconstruction bien au-delà), et la PLNE (HiGHS) **ne termine pas** en 60 s sur 4 d’entre elles. Sur ces 4 instances, **toutes les variantes d’AG, en ≈ 7 s, font mieux que la PLNE en 60 s** :

Instance (Jookan)	AG standard	QL pré-entraîné	PLNE 60 s	Glouton
n = 1 000, g = 10, s = 100	0,0007	0,0001	0,0003	0,2464
n = 1 000, g = 10, s = 300	0,0021	0,0020	0,0253	0,4662
n = 400, g = 10, s = 100	0,0027	0,0023	0,0092	1,0400
n = 400, g = 10, s = 300	0,0012	0,0011	0,0013	0,2244

(écarts en %, moyenne de 10 exécutions pour les AG). C'est la situation où une métaheuristique garde tout son intérêt face aux méthodes exactes (H4), comme le soulignent les revues récentes sur le sac à dos (CACCHIANI et al., 2022a; WILBAUT et al., 2022).

Figures associées : fig02_e2_ecart_par_classe, fig02_e2_rangs_friedman, fig02_e2_boites, fig02_e2_convergence, fig02_e2_succes_echecs, fig02_e2_qualite_temps.

5 Cas d'usage réels (données synthétiques illustratives; 5 exécutions par variante d'AG)

Cas	Glouton	PD	PLNE	AG standard	Aléatoire	QL en ligne	QL pré-entraîné
Budget communal (KP, n = 120)	0,073 %	non applicable (coûts décimaux)	optimal, 0,07 s	0,003 %	0 %	0 %	0 %
Camion humanitaire (MKP, m = 2, n = 250)	2,05 %	non applicable	optimal, 0,3 s	0,096 %	0,070 %	0,061 %	0,061 %
Machines virtuelles (MKP, m = 3, n = 300)	25,4 %	non applicable	optimal, 1,5 s	2,29 %	4,05 %	2,84 %	3,36 %
Portefeuille (KP corrélé, n = 400)	0,173 %	optimal, 0,17 s	optimal, 1,7 s	0,0045 %	0,0019 %	0,0022 %	0,0014 %

Ces cas reprennent des applications classiques du sac à dos : sélection de projets sous contrainte budgétaire, chargement multi-ressources, allocation de ressources informatiques et choix de portefeuille (KELLERER et al., 2004a; VAEZI et al., 2020; WILBAUT et al., 2022).

- Sur ces tailles modestes, la **PLNE** est la méthode de choix (optimale en moins de 2 s).
- Le **glouton** est très fragile en multidimensionnel (25 % d'écart sur les machines virtuelles), faute d'un ordre unique des efficacités lorsque plusieurs ressources sont contraintes (KELLERER et al., 2004b).
- Le QL pré-entraîné est le meilleur des AG sur 3 cas sur 4; il **échoue** sur les **machines virtuelles**, où l'AG standard, moins disruptif, fait mieux (voir la discussion).

6 E3 — Sac à dos multidimensionnel, OR-Library (58 instances × 6 variantes × 10 exécutions)

Ce jeu, issu de l'OR-Library (BEASLEY, 1990; CHU & BEASLEY, 1998), teste le **transfert inter-problèmes** : la Q-table Q_OOD, apprise sur le KP 0/1, est appliquée **sans modification** au MKP ($m = 5$ ou 10 contraintes). Référence : optimum SAC-94 (*weing*) ou meilleure valeur connue (PLNE 60 s ou meilleure exécution).

Tableau 4 — Écart moyen à la référence (%) :

Sous-ensemble	AG standard	Aléatoire	UCB1	QL en ligne	QL pré-entraîné	Oracle fixe	PLNE 60 s	Glouton
OR5x100 (30)	0,252	0,164	0,161	0,173	0,159	0,127	0,000	3,56
OR10x100 (10)	0,808	0,578	0,588	0,606	0,629	0,526	0,037	7,43
OR5x250 (10)	0,345	0,229	0,224	0,230	0,211	0,184	0,015	4,39
SAC-94 <i>weing</i> (8)	0,041	0,025	0,036	0,043	0,062	0,025	0,000	2,11
Moyenne	0,335	0,228	0,228	0,240	0,236	0,191	—	—
Rang moyen (Friedman)	5,55	3,36	3,17	3,69	3,29	1,93	—	—

Friedman : $\chi^2 = 128,2$, $p = 5,7 \cdot 10^{-24}$ ($N = 58$, $CD = 0,99$). Temps moyen par exécution d'AG : $\approx 1,9$ s.

- **H1 confirmée sur le MKP** : toutes les variantes adaptatives battent l'AG standard (QL pré-entraîné : p Holm $< 10^{-4}$, 28/30/0; QL en ligne : 22/36/0).
- **Le transfert inter-problèmes n'apporte pas de gain supplémentaire** : QL pré-entraîné, aléatoire, UCB1 et QL en ligne ne se distinguent pas (p Holm $\geq 0,41$; seul UCB1 est très légèrement meilleur que le QL en ligne, $p = 0,037$). La politique apprise sur le KP 0/1 conserve la diversification mais ne capture pas la structure multi-contraintes, que les travaux dédiés au MKP intègrent explicitement dans l'état ou dans les caractéristiques de l'instance (BUSHAJ & BÜYÜKTAHTAKIN, 2024; CACCHIANI et al., 2022b; REZOUG et al., 2022). L'**oracle** (UX + BF3) reste significativement meilleur (0/51/7) : le croisement uniforme est pertinent pour le MKP, mais la politique transférée l'utilise moins systématiquement (en phase intermédiaire notamment).
- **Méthodes exactes** : la PLNE (HiGHS) prouve l'optimalité sur 38 instances et atteint sa limite de 60 s sur 20 (9 OR10x100, 10 OR5x250, 1 OR5x100), avec un écart résiduel de 0,015 à 0,037 %. À ces tailles, **la PLNE en 60 s reste nettement meilleure que l'AG en ≈ 2 s**; l'AG offre un compromis qualité/temps (0,2 % en 2 s) et le **glouton** est très médiocre en multidimensionnel (2 à 7 %).

Figures associées : fig03_e3_ecart_par_classe, fig03_e3_rangs_friedman, fig03_e3_succes_echecs, fig03_e3_qualite_temps.

7 E5 — Passage à l'échelle (classes UC et SC, $n = 100$ à 10 000, 3 graines)

Tableau 5 — Temps moyen (s) et écart à la meilleure valeur trouvée (%) (budget de l'AG fixe : 200 générations × 100 individus) :

Classe	n	PD	PLNE	Glouton	AG standard	QL pré-entraîné
SC	1 000	1,2 s · 0 %	80 s* · 0 %	0,01 s · 0,134 %	2,5 s · 0,166 %	3,0 s · 0,021 %
SC	2 000	5,8 s · 0 %	120 s* · 0 %	0,01 s · 0,080 %	5,5 s · 0,270 %	5,9 s · 0,036 %
SC	10 000	infaisable	120 s* · 0 %	0,06 s · 0,017 %	15,8 s · 1,170 %	21,9 s · 0,365 %
UC	1 000	1,5 s · 0 %	0,5 s · 0 %	0,01 s · 0,011 %	4,1 s · 0,005 %	4,5 s · 0,002 %
UC	10 000	infaisable	33 s · 0 %	0,06 s · 0,0001 %	36 s · 6,25 %	40 s · 1,44 %

* limite de temps atteinte sans preuve d’optimalité. « Infaisable » : $n \cdot C > 3 \cdot 10^9$ opérations (garde-fou ; la PD dépasserait plusieurs minutes et plusieurs gigaoctets pour reconstruire la solution).

- La **PD** croît en $O(nC)$, soit $O(n^2)$ ici ($C \propto n$) : 6 s à $n = 2\,000$, hors de portée au-delà de 5 000. Elle est pseudo-polynomiale : le KP 0/1 n’est NP-difficile qu’au sens faible (GAREY & JOHNSON, 1979 ; KELLERER & PFERSCHY, 2004).
- La **PLNE** trouve la meilleure valeur à toutes les tailles, mais **ne prouve plus l’optimalité** sur la classe SC dès $n = 500$ (limite de 80–120 s).
- À budget fixe, **l’AG se dégrade avec la taille** ; le **QL pré-entraîné reste 3 à 8 fois meilleur que l’AG standard** jusqu’à $n = 10\,000$ — soit 50 fois la taille des instances d’entraînement : le transfert en taille fonctionne.
- **Échec à grande échelle** : à $n \geq 5\,000$, le **glouton** (0,06 s) fait mieux que toutes les variantes d’AG : avec 200 générations et une population initiale aléatoire, l’AG n’a pas le temps de converger sur 10 000 variables, alors que la solution gloutonne est quasi optimale lorsque n est grand (C grand devant chaque poids), propriété liée à la borne de Dantzig (DANTZIG, 1957 ; MARTELLO & TOTH, 1990). Remèdes évidents, non testés faute de temps : **initialisation gloutonne** d’une partie de la population (option `init_greedy_fraction` déjà implémentée) et **budget proportionnel à n** .

Figures associées : fig07_scalabilite.

Bibliographie

- AUER, P., CESA-BIANCHI, N., & FISCHER, P. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning*, 47(2–3), 235–256. <https://doi.org/10.1023/A:1013689704352>
- BEASLEY, J. E. (1990). OR-Library : distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11), 1069–1072. <https://doi.org/10.1057/jors.1990.166>
- BELLMAN, R. (1957). *Dynamic Programming*. Princeton University Press.
- BENJAMINS, C., CENIKJ, G., NIKOLIKJ, A., MOHAN, A., EFTIMOV, T., & LINDAUER, M. (2024). Instance Selection for Dynamic Algorithm Configuration with Reinforcement Learning : Improving Generalization. *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO ’24)*.
- BIEDENKAPP, A., BOZKURT, H. F., EIMER, T., HUTTER, F., & LINDAUER, M. (2020). Dynamic Algorithm Configuration : Foundation of a New Meta-Algorithmic Framework. *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI 2020)*, 427–434. <https://doi.org/10.3233/FAIA200122>
- BUSHAJ, S., & BÜYÜKTAHTAKIN, İ. E. (2024). A K-means Supported Reinforcement Learning Framework to Multi-dimensional Knapsack. *J Glob Optim.* <https://doi.org/10.1007/s10898-024-01364-6>

- CACCHIANI, V., IORI, M., LOCATELLI, A., & MARTELLO, S. (2022a). Knapsack problems — An overview of recent advances. Part I : Single knapsack problems. *Computers & Operations Research*, 143, 105692. <https://doi.org/10.1016/j.cor.2021.105692>
- CACCHIANI, V., IORI, M., LOCATELLI, A., & MARTELLO, S. (2022b). Knapsack problems — An overview of recent advances. Part II : Multiple, multidimensional, and quadratic knapsack problems. *Computers & Operations Research*, 143, 105693. <https://doi.org/10.1016/j.cor.2021.105693>
- CHU, P. C., & BEASLEY, J. E. (1998). A Genetic Algorithm for the Multidimensional Knapsack Problem. *Journal of Heuristics*, 4(1), 63-86. <https://doi.org/10.1023/A:1009642405419>
- DA COSTA, L., FIALHO, Á., SCHOENAUER, M., & SEBAG, M. (2008). Adaptive operator selection with dynamic multi-armed bandits. *Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation (GECCO '08)*, 913-920. <https://doi.org/10.1145/1389095.1389272>
- DANTZIG, G. B. (1957). Discrete-Variable Extremum Problems. *Operations Research*, 5(2), 266-288. <https://doi.org/10.1287/opre.5.2.266>
- DEMŠAR, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1-30.
- DERRAC, J., GARCÍA, S., MOLINA, D., & HERRERA, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1), 3-18. <https://doi.org/10.1016/j.swevo.2011.02.002>
- EIBEN, Á. E., HINTERDING, R., & MICHALEWICZ, Z. (1999). Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3(2), 124-141. <https://doi.org/10.1109/4235.771166>
- GAREY, M. R., & JOHNSON, D. S. (1979). *Computers and Intractability : A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- GOLDBERG, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- HOLLAND, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- HUANGFU, Q., & HALL, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119-142. <https://doi.org/10.1007/s12532-017-0130-5>
- JOOKEN, J., LEYMAN, P., & DE CAUSMAECKER, P. (2022). A new class of hard problem instances for the 0-1 knapsack problem. *European Journal of Operational Research*, 301(3), 841-854.
- KARIMI-MAMAGHAN, M., MOHAMMADI, M., MEYER, P., KARIMI-MAMAGHAN, A. M., & TALBI, E.-G. (2022). Machine learning at the service of meta-heuristics for solving combinatorial optimization problems : A state-of-the-art. *European Journal of Operational Research*, 296(2), 393-422. <https://doi.org/10.1016/j.ejor.2021.04.032>
- KELLERER, H., & PFERSCHY, U. (2004). Improved Dynamic Programming in Connection with an FPTAS for the Knapsack Problem. *Journal of Combinatorial Optimization*, 8(1), 5-11. <https://doi.org/10.1023/B:JOCO.0000021934.29833.6b>
- KELLERER, H., PFERSCHY, U., & PISINGER, D. (2004a). *Knapsack Problems*. Springer. <https://doi.org/10.1007/978-3-540-24777-7>
- KELLERER, H., PFERSCHY, U., & PISINGER, D. (2004b). Multidimensional Knapsack Problems. In H. KELLERER, U. PFERSCHY & D. PISINGER (Éd.), *Knapsack Problems* (p. 235-283). Springer. https://doi.org/10.1007/978-3-540-24777-7_9
- LI, P., HAO, J., TANG, H., FU, X., ZHENG, Y., & TANG, K. (2025). Bridging Evolutionary Algorithms and Reinforcement Learning : A Comprehensive Survey on Hybrid Algorithms. *IEEE Transactions on Evolutionary Computation*, 29(5), 1707-1728. <https://doi.org/10.1109/TEVC.2024.3443913>
- MARTELLO, S., & TOTH, P. (1990). *Knapsack Problems : Algorithms and Computer Implementations*. John Wiley & Sons.
- MNIH, V., KAVUKCUOGLU, K., SILVER, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533. <https://doi.org/10.1038/nature14236>

- PISINGER, D. (2005). Where are the hard knapsack problems? *Computers & Operations Research*, 32(9), 2271-2284. <https://doi.org/10.1016/j.cor.2004.03.002>
- REZOUQ, A., BADER-EL-DEN, M., & BOUGHACI, D. (2022). Application of Supervised Machine Learning Methods on the Multidimensional Knapsack Problem. *Neural Process Lett*, 54(2), 871-890. <https://doi.org/10.1007/s11063-021-10662-z>
- SONG, Y., WU, Y., GUO, Y., YAN, R., SUGANTHAN, P. N., ZHANG, Y., PEDRYCZ, W., CHEN, Y., DAS, S., MALLIPEDDI, R., & AJANI, O. S. (2023, août). Reinforcement Learning-assisted Evolutionary Algorithm : A Survey and Research Opportunities. <https://doi.org/10.48550/arXiv.2308.13420>
- SUTTON, R. S., & BARTO, A. G. (2018). *Reinforcement Learning : An Introduction* (2^e éd.). MIT Press.
- VAEZI, F., sadjadi seyed jafar , s. j., & MAKUI, A. (2020). Mean-VaR Portfolio Optimization Based on the Improved Knapsack Problem : Parametric and Nonparametric Approaches. *IJES*, 9(2). <https://doi.org/10.22099/ijes.2021.38133.1695>
- VARGHA, A., & DELANEY, H. D. (2000). A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2), 101-132. <https://doi.org/10.3102/10769986025002101>
- VIRTANEN, P., GOMMERS, R., OLIPHANT, T. E., et al. (2020). SciPy 1.0 : Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- WATKINS, C. J. C. H., & DAYAN, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292. <https://doi.org/10.1007/BF00992698>
- WILBAUT, C., HANAFI, S., COELHO, I. M., & LUCENA, A. (2022). The Knapsack Problem and Its Variants : Formulations and Solution Methods. In S. SALHI & J. BOYLAN (Éd.), *The Palgrave Handbook of Operations Research* (p. 105-151). Springer International Publishing. https://doi.org/10.1007/978-3-030-96935-6_4
- WOLPERT, D. H., & MACREADY, W. G. (1997). No Free Lunch Theorems for Optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67-82. <https://doi.org/10.1109/4235.585893>